PROXY WAR • NOTEBOOK CONTEXT

Could “Let’s Play” fit Proxy War?

A concise view of the existing agent path and why the format feels relevant

Short answer

Yes—the guided format looks relevant to Proxy War. The material in this packet is shared only as context for that reaction. It is not a specification, assignment, or assumption that you want to build it.

WHAT EXISTS	WHY IT MAPS	WHAT IS OPEN
A Coworld starter policy, legal-action and replay contracts, canonical validation, and match evidence.	The CrewRift flow—understand, edit, test, run, replay, improve—matches the natural Proxy War agent-learning loop.	Whether this is interesting, what form it might take, and whether any follow-up is useful are deliberately left open.

This packet contains a short overview plus a few deliberately selected public examples. Operational commands, live league identifiers, internal project notes, and time-sensitive hosted claims are intentionally excluded.

REPOSITORY-VERIFIED / STABLE PRODUCT CONTEXT

1. Proxy War in 60 seconds

Proxy War is an autonomous strategy game in which software agents claim territory, build an economy, form alliances, attack rivals, and play complete matches without human input. At each decision, the game gives a policy the current observation and a list of actions that are legal at that moment.

The durable contract

A policy returns one exact LegalAction.id from the list it was offered. It does not construct a raw game-engine intent. The existing validator, runner, and GameServer remain authoritative.

2. What already exists

Material	What it contributes	Status in this packet
Coworld starter policy	A small rule-based agent with one clearly marked strategy function.	Included as the simplest working reference.
Player protocol	The minimal decision_request → decision_response contract.	Included as a reviewed reference copy.
Global and replay protocol	The status, spectator, result, and saved-replay surfaces.	Included without time-sensitive league details.
Canonical match path	Validation, episode execution, results, and replay artifacts.	Explained here; live hosted details intentionally omitted.

One architecture, not a notebook-only SDK

Any notebook-shaped experience would be most useful if it exposed the existing policy and match path. It should not introduce a second protocol, validator, runner, or raw-intent route.

PROPOSED / EXPLORATORY MAPPING

3. How the format might translate

The strongest overlap is the learning sequence, not any specific CrewRift implementation detail. A Proxy War version could, in principle:

- Show a replay or match outcome first, so the learner understands the destination.
- Open one frozen decision request and explain the observation plus offered legal actions.
- Run a working baseline policy, then change one small piece of strategy or selection logic.
- Verify that the policy selected one offered LegalAction.id through the canonical path.
- Inspect replay/result evidence and compare one controlled change with the baseline.

> **No implied ask**
>
> This is simply the part of the CrewRift notebook that appears transferable. It is not a proposed scope, commitment, or request for you to take ownership of it.

4. What is included

File	Purpose
README_FIRST.md	Packet orientation and current-use caveats.
SEND_WITH_THIS.txt	A short, neutral message to accompany the packet.
ProxyWar_Notebook_Context.docx	This concise overview.
protocol/	The player and global/replay contracts.
reference-agent/	A small non-LLM Coworld policy baseline.
licenses/	Applicable license copies for the selected source material.

5. Public source links

Public material	Link and use
Proxy War source	github.com/0xNad/ProxyWar ↗ Platform and protocol source of truth.
Proxy War Coworld adapter	github.com/0xNad/ProxyWar/tree/main/coworld-adapter ↗ Active Coworld integration and the source of the selected examples.
Coworld source	github.com/Metta-AI/coworld ↗ Upstream packaging and league tooling.